

VLASelect: Selective Large-small Model Co-learning for Self-evolving VLA Agents

Anonymous Author(s)

Abstract

VLA (Vision-Language-Action) model-based agents such as robot arms, dexterous hands and humanoid robots are revolutionizing our lives. These agents usually run in open ended, interactive environments where new tasks start, surroundings change, or available resources fluctuate. In existing agentic AI systems, the resource-intensive training of deployed VLA models has become a critical bottleneck. In this paper, we present VLASelect, a framework that takes the dynamics of agents' interactive environment into account to enable large-small model collaborative learning. In online RL, VLASelect employs an agent's small model to quickly explore the environment, selectively transfers its positive knowledge to the agent's large model, and compensates its learning ability by swapping in the large model's most accuracy-related neurons. In doing so, our approach combines the strengths of large models' high learning capacity and small models' low training costs via neuron-grained knowledge exchange. Our experiments show that compared to 9 state-of-the-art VLA learning techniques across 4 embodied AI agents, VLASelect achieves as much as 40.12% increase in task success rate, 25.6% decrease in memory footprint and 11.55x reduction on energy consumption.

CCS Concepts: • Human-centered computing → Ubiquitous and mobile computing; • Computing methodologies → Neural networks.

Keywords: Embodied AI systems, VLA, self-evolving agents, large-small model co-learning

ACM Reference Format:

Anonymous Author(s). 2027. VLASelect: Selective Large-small Model Co-learning for Self-evolving VLA Agents. In *Proceedings of European Conference on Computer Systems (EuroSys '27)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. EuroSys '27, Rabat, Morocco

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

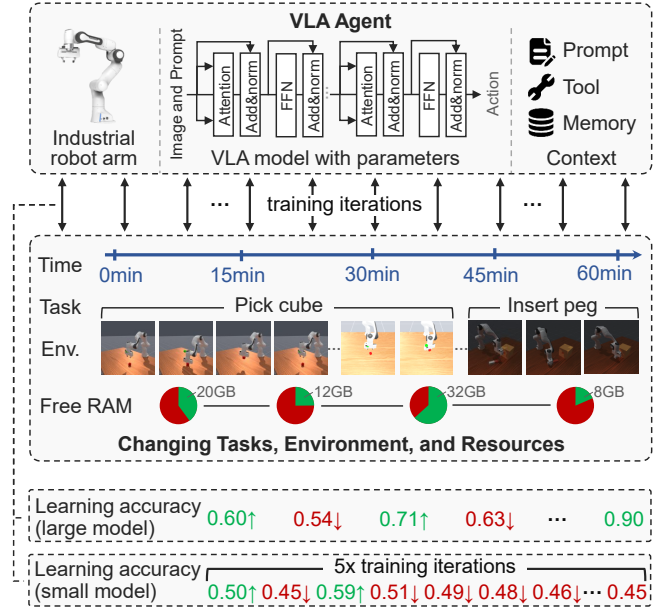


Figure 1. Scenario of self-evolving VLA agents.

1 Introduction

Vision-language-action (VLA) agents have become ubiquitous in embodied AI applications [39, 49], spanning from intelligent manufacturing [15, 32] to smart homes [6] to humanoid robots [63]. Today's embodied AI agents are deployed in open-ended, interactive environments, termed **self-evolving agents**, in which agents must autonomously optimize their components for novel tasks (e.g. from picking cube to inserting peg in Figure 1), changing environment (e.g. lights or visual angles) or fluctuating computational resources (e.g. RAM) [16]. While in-context learning (ICL) (updating non-parametric components such as prompts, memory, and toolset) works well for large language models (LLMs) [12, 17], VLA models have no inherited ICL ability [54] and usually rely on resource-intensive reinforcement learning (RL) [9, 22, 29, 50] to learn from environment feedback. The stringent computation, memory, and power constraints of agent devices [7] hinder the training of large VLA models with high learning capacity. The challenge of coping with resource is further exacerbated when encountering RL's time-consuming exploration process.

Limitations of learning with fixed models. The key to achieving the full promise of successful agent evolution is to efficiently update VLA models in correspondence to the changing environment. Existing VLA learning techniques

including online RL [9, 19, 22, 35, 51] and model-based RL [29, 62] rely on *fixed network architectures* such as Figure 1’s transformer-based VLA model. At one *training iteration*, they employ the whole model to perform the forward operations in sample steps and update parameters in trainable layers such as attention/FFN layers in the training step. However, treating all VLA parameters equally in on-device RL gives rise to two major challenges.

Low accuracy returns on resource investment. In ever changing scenarios, large VLA models provide high abilities in learning knowledge but also consume large computational resources. More importantly, the sparsity nature of neural network [20, 38, 53, 66] means only *a small percentage of neurons are activated and contribute to accuracy* at one training iteration. As a result, equally updating parameters in trainable layers such as attention/FFN layers wastes most of resources. **The first challenge** now requires us to depart from the equal treatment of model parameters and investigate how to selectively invest available resources on the most accuracy-related model parameters.

Insufficient environment exploration. VLA agents take long time to explore the environment with RL. As illustrated in Figure 1, the small model completes 5x training iterations within the same training time, but only achieves accuracy improvement in its early iterations because of plasticity (learning ability) loss [11]. In contrast, the large model completes much fewer iterations and intermittently has accuracy increases across iterations. **The second challenge**, therefore, is how to combine the strengths of both small and large models in a way that quickly explores the environment using the small model, while keeping its learning ability and acquired knowledge based on the large model.

In this paper, we present VLASelect, a framework designed fundamentally based on network neurons (a basic unit of neural networks [10, 58]) that widely exist in VLA models of today’s agentic AI systems. The key idea of VLASelect is to **selectively exchange neurons** between the large and small model during the online RL process. This exchange mechanism allows the small model to dynamically swap in the large model’s most accuracy-related neurons according to the environment and system status and thus maintains its learning ability. Utilizing this small model in RL, VLASelect also selectively transfers its learned knowledge to the large model by swapping out the accumulated neuron updates. In doing so, VLASelect combines the strengths of both large models of high learning capacity and small models of low training overheads. In particular, the contributions of this paper are as follows:

- **Tailored small model to changing environment.** VLASelect comprehensively considers new tasks, environment changes, and available resources, and customizes the size and architecture of the small model to retain the most accuracy-related neurons.

- **Fast exploration with knowledge accumulation and plasticity compensation.** VLASelect employs the small model to fast explore the environment, selectively transfer its learned knowledge to the large model without direct training, while maintaining its learning ability by swapping in the large model’s neurons.
- **Implementation and evaluation.** We implement VLASelect on commodity robotic computing platforms, and conduct extensive evaluation against 9 state-of-the-art VLA learning techniques on 4 representative VLA agents (robotic arm, dexterous hand, mobile manipulator, and humanoid).

Summary of experimental results. We extensively compare VLASelect against state-of-the-art (SOTA) techniques using prevalent embodied AI benchmarks. These benchmarks cover 4 embodied AI agents (single-arm robot, dexterous hand, mobile manipulator, and humanoid robot) and new tasks, surroundings changes, and available resource fluctuations in online RL scenarios. The results show: (i) VLASelect provides 40.12% higher accuracies compared to 9 competing baselines with the same resource budget; (ii) VLASelect saves time cost, memory footprint, and energy consumption by 11.49x, 1.34x (25.6%), and 11.55x with the same learning accuracy; and (iii) we fully test VLASelect’s design choices, validate its sim-to-real performance, and discuss its limitations (e.g. maximum supported model sizes) when applied in resource-constrained agentic AI devices.

2 Motivation

In this section, we lay out several observations to understand the network neuron in VLA learning, and gain insights on the limitation of fixed-model based training.

Neurons in VLA. The limitation of existing approaches is rooted in the constraint that they provide a coarse-grained differentiation of resources in VLA model training. Thus, this work studies a more detailed VLA model unit: network neuron. As illustrated in Figure 2, a **neuron** [10, 58] is a fundamental computational unit that stores a row of model parameters and outputs a channel of feature map. A transformer layer consists of several hundreds to one thousand neurons.

Limitation 1: Highly unbalanced neuron contributions to model accuracy. In VLA model training, a neuron’s **contribution to accuracy** represents the accuracy drop when it is removed from the model [41, 67] and this value can be calculated by auxiliary network modules (e.g. Feature Boosting Suppression (FBS) [18] or Utility [14]). In Figure 2’s example, we apply the PPO [50] to train a large VLA-Adapter [59] model and a small one using the Maniskill [55] benchmark. Although the large model has 13.7% higher accuracy than the small one, we can see that this model only activates a small percentage of its neurons: about

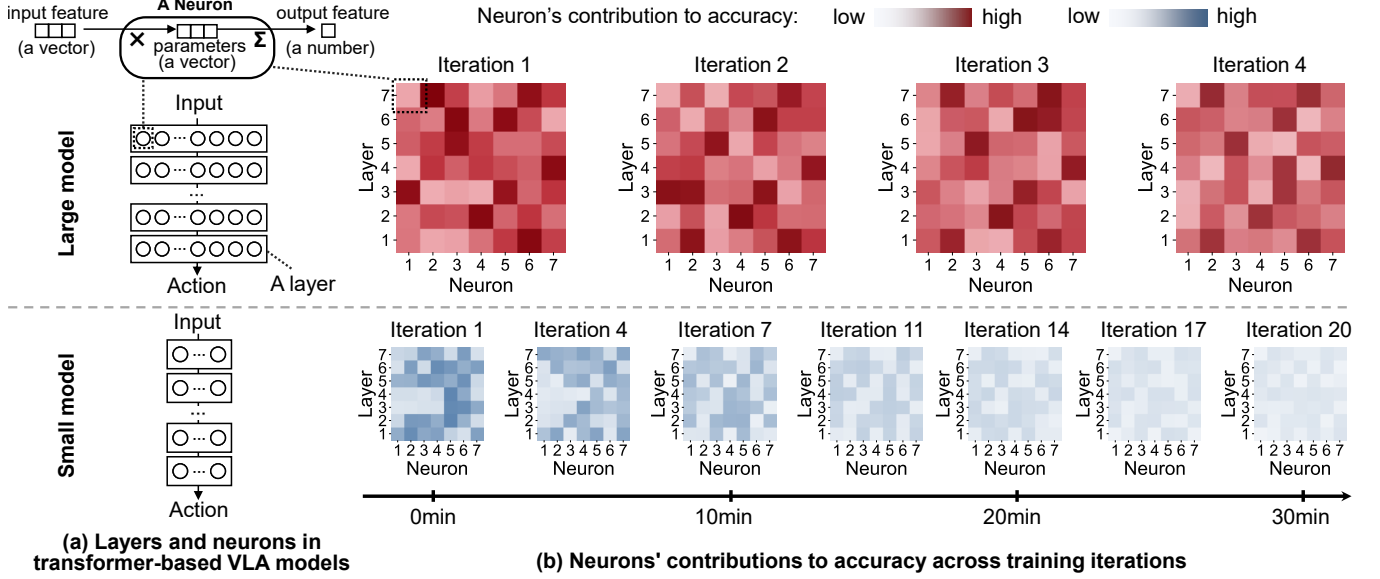


Figure 2. Motivation of large-small model co-learning. We demonstrate 7 neurons in 7 model layers as an example.

20% of neurons have much higher contributions than other neurons.

Research question 1: *In response to task, environment and resource changes, how to perform neuron-grained search of the optimal network on the fly?*

Limitation 2: Conflict of exploration effectiveness and efficiency. In online RL, utilizing the large model in exploration (i.e. sampling and model training) [50] achieves higher model accuracy. This is because this model has more neurons with high accuracy contributions, as illustrated in Figure 2. In contrast, the small model has high exploration efficiency. In Figure 2’s example, this model completes 5x more training iterations, but the accuracy contributions of its neurons decrease quickly, causing its learning ability loss.

Research question 2: *How to enable neuron exchanges between the large and small models to achieve both high effectiveness and efficiency in online RL?*

3 Design of VLASelect

3.1 Overview

Figure 3 illustrates the architecture of VLASelect with three modules. At each training iteration, they work together to enable large-small model co-learning with four steps.

At step 1, VLASelect triggers the *optimal network searcher* (§3.2) to output the small model’s optimal network architecture according to the current environment and available resources. To this end, the searcher first encodes each network architecture’s input data and all layers’ neuron numbers into a scaling law function [26], and predicts this network’s accuracy. This searcher then performs a heuristic search of different network architectures and quickly finds the one with the highest accuracy.

At step 2, VLASelect employs the *selective model enhancer* (§3.3) to scale up or down the current small model according to step 1’s optimal network architecture. Specifically, the enhancer first increases or decreases the number of neurons in each network layer, and then swaps in the large model’s most accuracy-related neurons to enhance the small model’s learning ability. This is achieved by constructing *neuron indexes* among corresponding neurons in the large and small models, thus supporting online model scaling without retraining.

At step 3, VLASelect trains the small model with one or multiple training iterations, and only triggers step 4 (e.g. after training iteration k or n in Figure 3) when this model accumulates sufficient model updates for a positive knowledge transfer. Finally, at step 4, the *selective knowledge accumulator* (§3.4) selectively swaps out the small model’s accumulated knowledge to the large model, thus allowing the later model to continuously learn from the environment without direct training.

3.2 Optimal network searcher

The searcher finds the network architecture that maximizes the small model’s learning accuracy for the current input distribution and available resources. This is achieved by first constructing a scaling law function to predict the model accuracy and then calibrating the function whenever model or environment changes. Based on the predicted accuracies, the searcher develops a heuristic search to iterate over all possible network architectures and quickly prune most of unnecessary search space.

Scaling law construction. Traditional scaling law function [26] is designed for model pre-training and predicts model accuracy according to model size, dataset size, and

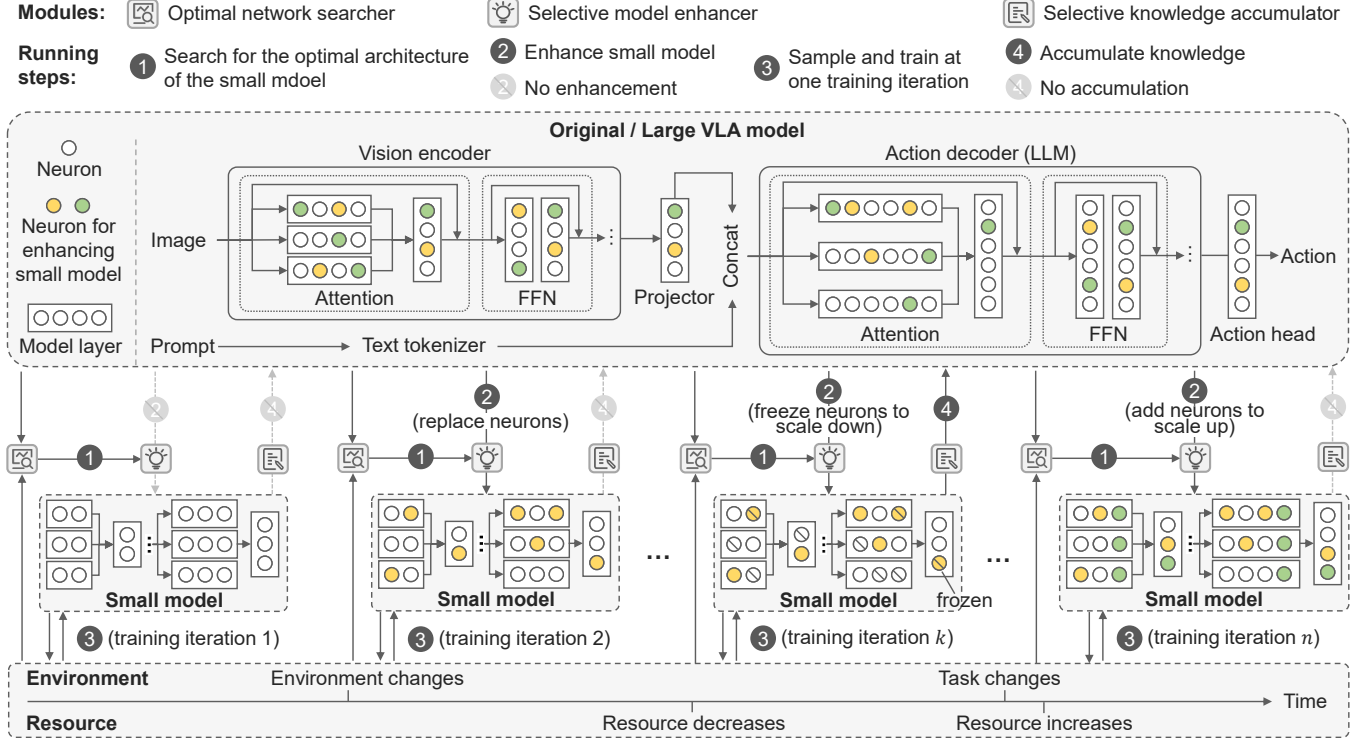


Figure 3. Overview of VLASelect.

available resources. In VLASelect, the searcher further encodes the large and small models' current architectures and input distributions into the function, and outputs the learning accuracy per unit of resources A :

$$A = f(w_1, w_2, \dots, w_L, r, \beta, \gamma) \\ = \prod_{i=1}^L \beta_i(w_i)^{\gamma_i} \cdot \beta_{L+1} r^{\gamma_{L+1}}$$

The input arguments can be divided into three categories:

- *Factors of current small model:* w_i is the number of neurons in the i -th layer ($1 \leq i \leq L$). It is proportional to the model size in training, and thus has a power-law relationship with the accuracy [26].
- *Factors of available resources:* r is the amount of available resources. It also has a power-law relationship with the accuracy [26].
- *Factors of current large model and input distribution:* β and γ jointly represent the impact of the current large model and input distribution on the accuracy: (i) β is the accuracy contribution of the large model's neurons; and (ii) γ is the current distribution's closeness to the source distribution. Both factors are positively correlated with the accuracy [5, 37].

Coefficient calibration. The searcher develops the calibration based on two theories. First, the gradient flow theory [37] states that model accuracy is positively correlated

with the neuron's contribution to accuracy. Second, the domain adaptation theory [5] states that model accuracy is inversely correlated with the discrepancy between current distribution and source distribution. The searcher constructs a tiny neural network to jointly fit these two correlations. The network takes the large model's neuron contribution and distribution discrepancy as input, and outputs the calibrated coefficients β and γ .

Optimal network architecture search. The searcher aims to find the values of w_1 to w_L that maximize the output of calibrated scaling law. It first initializes all element w_1 to w_L to 0, and then searches for their optimal values through multiple iterations. At each iteration, it calculates the accuracy improvement of increasing each element w_i ($1 \leq i \leq L$) by 1. Subsequently, the searcher selects the element bringing the largest accuracy improvement, increases it by 1, and reduces the available resources by one unit. The search stops when the available resources are reduced to 0. In a word, the search follows a heuristic greedy strategy: it first adds the neuron bringing the largest accuracy improvement to the optimal architecture.

Overhead analysis. In the searcher, each prediction of the scaling law function involves several multiplication/power-law computations and completes within 0.01 ms. The network for coefficient updating is less than 1 MB and this calibration is only conducted once for a training iteration.

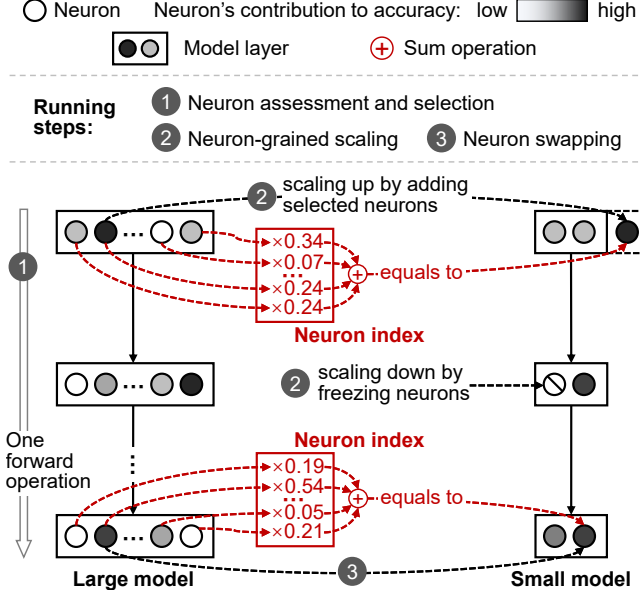


Figure 4. Steps of the selective model enhancer.

Hence, the whole heuristic search prunes over 99.9% of possible architectures and can complete within 0.1s.

3.3 Selective Model Enhancer

The enhancer selects the large model’s neurons to improve the small model’s learning ability, and it is developed based on the neuron index between the large and small models. As illustrated in Figure 4, a **neuron index** is a vector of weights $\{\Gamma_1, \Gamma_2, \dots\}$ that maps all neurons in the large model’s layer into one neuron in the small model. Specifically, in the small model, each neuron θ^{small} is a linear combination of all neurons $\{\theta_1^{\text{large}}, \theta_2^{\text{large}}, \dots\}$ in the large model’s layer: $\theta^{\text{small}} = \sum_i \Gamma_i \theta_i^{\text{large}}$, where Γ_i is the similarity between two neurons.

Based on the neuron index, the enhancer utilizes three steps to replace the small model’s neurons with the large model’s most accuracy-related ones.

Step 1: Neuron assessment and selection. This step first calculates neurons’ contributions to model accuracy by attaching a Feature Boosting and Suppression (FBS) module [18] to each layer of the large model. Subsequently, step 1 ranks neurons in each layer in descending order of their accuracy contribution and selects w_i top-ranked neurons as the most accuracy-related ones.

Step 2: Neuron-grained scaling. This step scales the small model using multiple iterations. At the i -th iteration, the enhancer scales the small model’s i -th layer with w'_i neurons to match the target size w_i . (i) *Scaling up*. If $w'_i < w_i$, the enhancer scales up the layer by adding $w_i - w'_i$ selected neurons from the large model’s i -th layer. These neurons retain previously learned knowledge and thus require no retraining. (ii) *Scaling down*. If $w'_i > w_i$, the enhancer scales

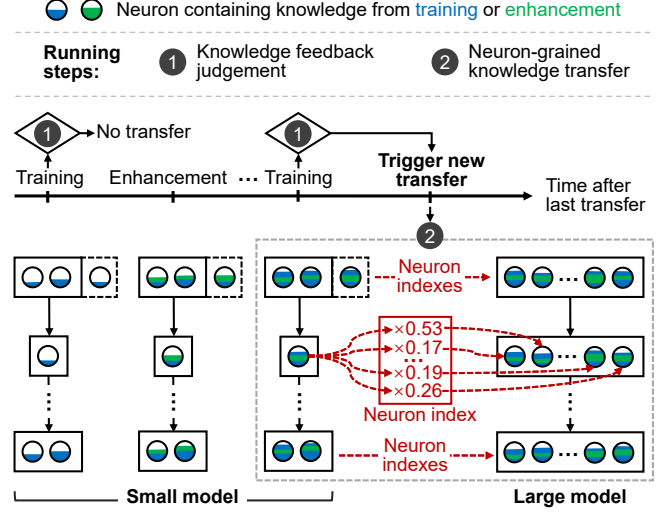


Figure 5. Steps of the selective knowledge accumulator.

down the layer by freezing $w'_i - w_i$ neurons that contribute least to accuracy. Freezing these neurons not only saves the resource consumption of backward operations, but also avoids the accuracy drop due to neuron pruning.

Step 3: Neuron swapping. This step aims to improve the small model’s learning capacity with multiple swapping iterations. At each iteration, the enhancer identifies the most accuracy-unrelated neuron in the small model, and replaces it with the large model’s neuron that (i) is linked to it with neuron index and (ii) has the highest similarity Γ_i with it. The swapping process completes when all selected neurons at step 1 are swapped into the small model.

Overhead analysis. In the enhancer, the calculation of all neurons’ accuracy contributions at step 1 only need one forward operation of the large model. Steps 2 and 3 involve memory operations. Taking VLA-Adapter with 0.5B parameters as an example, the enhancer occupies 50MB memory and its steps complete within 1s.

3.4 Selective Knowledge Accumulator

The accumulator enables the positive knowledge transfer between the small and large models via selective neuron exchanges. As illustrated in Figure 5, a neuron’s exchanged knowledge is its **accumulated parameter updates**. Between two knowledge transfers, these updates come from two sources: model training (i.e. gradients calculated in RL) and model enhancement (i.e. swapped in neurons from the large model). The accumulator has two steps.

Step 1: Knowledge feedback judgment. The accumulator filters out the knowledge and skips feedback if the small model has no statistically significant accuracy improvement: $A - \mu < 3\sigma$, where A denotes the small model’s current accuracy measured from the environment, μ and σ denote the mean and standard deviation of the small model’s historical

accuracies after last accumulation. A low improvement indicates that the small model has not learned sufficient positive knowledge.

Step 2: Neuron-grained knowledge transfer. Let $\Delta\theta^{\text{small}}$ be the accumulated parameter update of the neuron θ^{small} in the small model. The accumulator updates this neuron’s corresponding neuron in the large model as follows:

$$\theta_i^{\text{large}} \leftarrow \theta_i^{\text{large}} + \eta \cdot \Gamma_i \cdot \Delta\theta^{\text{small}}$$

The accumulator guarantees positive knowledge transfer in two ways. First, η denotes the normalized magnitude of $\Delta\theta^{\text{small}}$ among all neurons in the small model, representing the amount of new knowledge stored in $\Delta\theta^{\text{small}}$. Hence a larger η means $\Delta\theta^{\text{small}}$ should be more transferred into the large model. Second, Γ_i is the similarity between θ^{small} and θ_i^{large} recorded in the neuron index. That is, two similar neurons have similar functionality in sampling and similar parameter updating directions in training. Hence when updating θ_i^{large} , a higher similarity means transferring more parameter updates to the large model; otherwise, neuron θ_i^{large} retains more knowledge in the original model.

Overhead analysis. In the accumulator, step 1 involves a few basic mathematical operations such as additions and comparisons. Step 2 only performs memory copy and tensor addition operations. Both steps can complete within a few seconds.

4 Implementation

We implement VLASelect in Python 3.11.9 with 2k LoCs. As shown in Figure 6, we decouple the implementation into VLA modules, interfaces, and built-in system software.

Neuron swapping in VLASelect. Frequent neuron swapping operations are the major performance bottleneck of VLASelect’s modules. Our implementation improves this swapping in two ways. First, it submits neurons of different layers to different CUDA Streams and thus supports the parallel swapping of these neurons. Second, it stores neurons in LRU (Least Recently Used) Cache, which loads infrequently used neurons on demand to reduce memory usage.

Applicability. VLASelect implements three types of interfaces: (i) *VLA model*: this interface provides APIs for model initialization (`load_model()`) and operation (`forward()` and `backward()`). (ii) *Online RL algorithm*: this interface provide APIs for sampling (`env.sample()`) and environment reward computation (`env.step()`). (iii) *Agent*. This interface uses URDF (Universal Robot Description Format) and `do_action()` to define agents and their actions. Each class requires developers to write 50 to 100 LoCs.

Sim-to-real transfer. VLASelect implements three tools to support sim-to-real transfer of embodied AI agents: (i) *environment-level alignment* [25] aligns key physical parameters (e.g. friction coefficients and damping factors) between simulated environment and real environment. (ii) *RL-level*

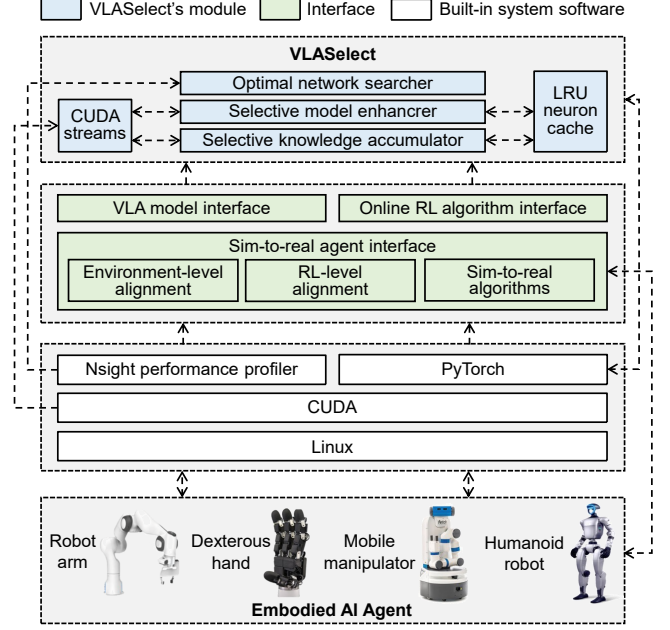


Figure 6. VLASelect’s implementation.

alignment [13] aligns online RL’s observation and action spaces with real embodied agent. (iii) *sim-to-real enhancement algorithms* improve the model’s generalization ability by injecting noise into the sampled data [56], imitating human’s behavior [47], or decoupling complex task into simple ones to learn sequentially [28]. Detailed information of these tools is presented in Appendix A of the supplement material.

5 Evaluation

5.1 Experimental Setup

Testbeds. We select three heterogeneous computing platforms in embodied AI agents: (1) NVIDIA AGX Xavier with 32GB memory and 512 Volta GPU cores; (2) NVIDIA AGX Orin with 32GB memory and 1792 Ampere GPU cores; and (3) NVIDIA AGX Orin with 64GB memory and 2048 Ampere GPU cores. They are equipped with Ubuntu 22.04 operating system and PyTorch 2.4.0.

Workloads. We choose four embodied AI applications, and construct workloads for self-evolving agents, as listed in Table 1.

- *Single-arm robot* consists of seven movable joints and one gripper. It takes images from the front camera as input, and picks up the target object (e.g. cube and sphere) on the table.
- *Dexterous hand* consists of four fingers and each finger has two movable joints. It takes images from the front camera as input, and grasps the target object with a complex shape (e.g. hammer and power plug).
- *Mobile manipulator* consists of a mobile base and a robotic arm. It takes images from both the head camera and the

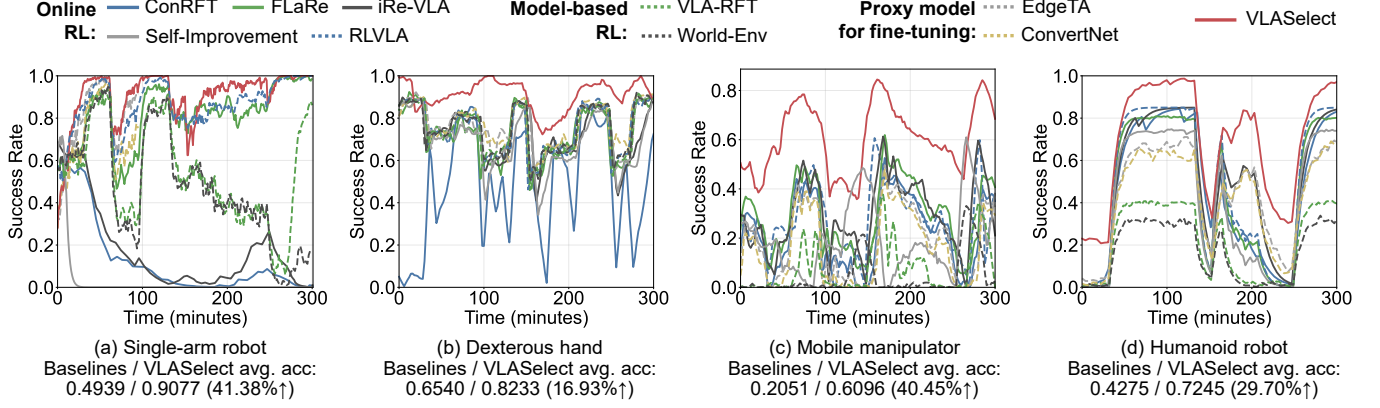


Figure 7. Accuracy comparison under new tasks and environment changes.

gripper camera as input, moves towards the cabinet, and opens the target drawer.

- *Humanoid robot* mimics the human body structure. It takes images from both the head camera and the front camera as input, grabs the target object with its right arm, and places it in a bowl to the side.

Metric. All workloads are implemented in ManiSkill benchmark, and use the *task success rate* as the accuracy metric.

Baselines. We compare VLASelect with three types of nine VLA learning techniques:

- *Online RL* explores the environment to collect training samples and update model parameters: (1) ConRFT (Consistency-based Reinforced Fine-Tuning) [9] retains the same loss structure from the offline stage and incorporates human interventions; (2) FLRe (Fine-Tuning Large-Scale Policies with RL) [22] introduces several strategies to stabilize RL training; (3) iRe-VLA (Iterate between RL and Supervised Learning) [19] collects success trajectories during RL and uses them for supervised learning; (4) Self-Improvement [51] predicts a better reward function by itself; (5) RLVA [35] develops a simple recipe for efficient PPO.
- *Model-based RL* uses world models to replace the interactive environment in online RL: (1) VLA-RFT [29] leverages a world model to predict future visual observations according to the given actions; (2) World-Env (World Model as a Virtual Environment) [62] integrates a video-based world simulator and a VLM-guided instant reflector to replace interactions with the environment.
- *Proxy model for fine-tuning* generates and fine-tunes a specific small proxy model on each given dataset: (1) EdgeTA (Edge Transformer Adaptation) [73] keeps the proxy model fixed during training; (2) ConvertNet [72] scales the proxy model once the available resource changes.

5.2 Comparison of Accuracy in Self-evolving Agents

In this evaluation, we compare the accuracy of VLASelect and baseline techniques under the same resource budget.

Table 1. Summary of applications and workloads.

Application	Robot	New tasks	Environment changes	VLA model
Single-arm robot	DOFBOT-SE [1]	Different objects, sizes and colors	Illumination, color temperature, and visual angle	Octo [40]
Dexterous hand	Amazing-Hand [2]	Different objects and sizes		VLA-Adapter [59]
Mobile manipulator	Fetch [3]	Different drawers and cabinets		TinyVLA [60]
Humanoid robot	UnitreeG1 [4]	Different objects and sizes		EdgeVLA [7]

5.2.1 Accuracy Under Tasks/Environment Changes.

Settings. At the offline stage, we pre-train an uncompressed original model for each workload. The pre-training uses the standard task and environment provided by ManiSkill, and is conducted on a NVIDIA A100 graphics card with 80GB VRAM. The pre-training time of baseline techniques ranges from 13.1 to 31.7 hours, and the time of VLASelect ranges from 9.3 to 29.8 hours. Model-based RL baseline techniques have the longest pre-training time due to their construction of world models. The pre-training is performed once for all approaches.

At the online stage, we run four workloads on different platforms: (i) single-arm robot and dexterous hand workloads run on NVIDIA AGX Xavier; (ii) mobile manipulator workload runs on NVIDIA AGX Orin with 32GB memory; and (iii) humanoid robot workload runs on NVIDIA AGX Orin with 64GB memory. Each workload runs for 5 hours and includes 10 randomly triggered events (new tasks or environment changes). Due to the limited memory on the platforms, the online RL and model-based RL techniques train the fixed compressed model whose size is 1/10 of the

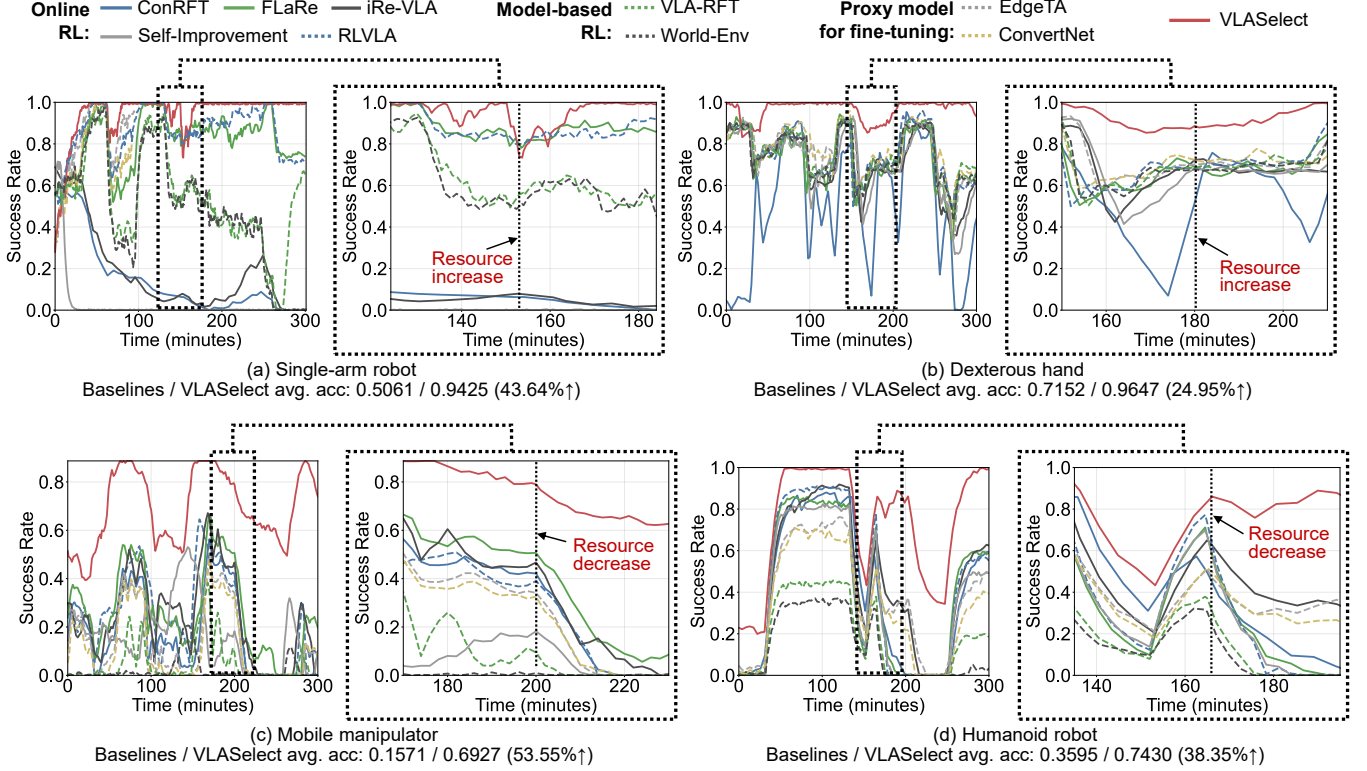


Figure 8. Accuracy comparison under new tasks, environment changes, and fluctuating available resources.

original model. EdgeTA, ConvertNet, and VLASelect construct the proxy/small model of the same size.

Results. Figure 7 demonstrates the accuracy of each method across four workloads. We have two key observations. First, VLASelect achieves the highest accuracy across all workloads. Second, its accuracy advantage becomes more pronounced after more training iterations (e.g. after 20 minutes in Figure 7(c)). This is because VLASelect allocates resources to the most accuracy-related neurons by optimal network search and selective neuron swapping.

In contrast, online RL and model-based RL baselines have lower accuracies, and some of them even suffer from continuously decreasing accuracy (e.g. ConRFT and iRe-VLA in Figure 7(a)). This is caused by the limited learning capacity of their models, whose fixed neurons may become saturated after continuously learning new knowledge. In addition, the proxy model for fine-tuning approaches also have lower accuracies. This is because the neurons of their proxy models are determined by a given dataset rather than dynamic environment, thus having similar saturation problems when applying in interactive RL.

Overall, VLASelect *improves accuracy by an average of 31.12% under new tasks and environment changes.*

Settings. This evaluation follows the settings of Section 5.2.1, and evaluates scenarios of available resource fluctuations. During each workload’s 5-hour evaluation period, we randomly trigger five changes in available resources (GPU cores and memory), each of which increases or decreases the available resources by 30%.

Results. Figure 8 presents the evaluation results. We can see that VLASelect still achieves the highest accuracy across all workloads. When available resources increase, VLASelect promptly improves the learning accuracy (e.g. Figure 8(a)) by swapping in additional neurons into the small model. Moreover, when available resources decrease, VLASelect mitigates the resulting accuracy degradation (e.g. Figure 8(d)) by freezing the least accuracy-related neurons.

In contrast, online RL and model-based RL baselines have lower accuracies for two reasons. First, their fixed models prevent fine-grained allocation of newly available resources. Second, they do not support dynamic scaling down, which results in insufficient environment exploration (e.g. fewer training iterations) when available resources decrease. In addition, ConvertNet dynamically scales model size but still has lower accuracies. This is because its network neurons are pre-determined by a given input data rather than interactive environment.

5.2.2 Accuracy Under Available Resource Changes.

Table 2. Overhead comparison under the same learning accuracy. "(a)" to "(d)" represent four workloads, respectively.

	Time (h)				Memory footprint (GB)				Energy consumption (kJ)			
	(a)	(b)	(c)	(d)	(a)	(b)	(c)	(d)	(a)	(b)	(c)	(d)
ConRFT	3.88	3.14	2.83	3.78	19.12	20.88	27.92	35.71	360.39	294.89	334.87	600.23
FlaRe	2.56	3.60	2.92	2.91	23.14	31.39	27.49	33.70	295.13	384.55	384.75	492.30
iRe-VLA	2.71	3.38	2.78	3.31	21.93	24.55	27.30	42.19	253.73	374.00	382.41	630.36
Self-Improvement	4.52	3.48	2.86	3.95	20.93	23.52	27.53	42.11	547.71	309.29	337.90	763.35
RLVLA	2.74	3.32	3.52	3.20	15.26	17.31	18.78	32.11	275.99	353.41	546.13	608.56
VLA-RFT	3.20	3.39	2.39	4.07	28.54	30.99	29.04	60.02	319.56	412.78	341.89	704.55
World-Env	3.10	3.63	3.87	4.16	25.66	30.05	29.27	60.23	279.42	429.66	505.86	738.14
EdgeTA	4.59	3.17	2.84	3.12	17.54	19.23	21.12	37.87	523.80	382.06	440.83	467.90
ConvertNet	4.82	3.67	2.69	2.73	15.26	17.31	18.78	32.11	478.14	434.77	430.10	510.83
VLASelect	0.39	0.29	0.21	0.32	15.26	17.31	18.78	32.11	45.53	34.53	28.76	47.56

Table 3. Average energy consumption (kJ) in each new task (first row of each method) and environment change (second row of each method).

	Single-arm obot	Dexterous hand	Mobile manipulator	Humanoid robot
ConRFT	342.37	346.31	499.84	787.19
	403.51	274.05	291.68	570.22
FlaRe	318.06	477.07	461.29	671.17
	254.18	317.51	379.37	452.06
iRe-VLA	383.33	530.53	505.4	661.88
	213.81	204.04	273.55	564.21
Self-Improvement	551.76	416.37	497.49	801.52
	473.14	293.83	297.81	673.49
RLVLA	326.26	360.71	573.44	622.44
	206.67	314.32	466.58	458.46
VLA-RFT	323.69	433.42	374.09	795.6
	277.8	280.17	257.39	632.21
World-Env	329.26	339.87	666.9	775.05
	265.45	451.14	480.57	639.55
EdgeTA	349.4	383.21	464.28	629.97
	549.99	282.31	355.5	444.5
ConvertNet	608.37	456.51	451.61	536.37
	454.23	327.35	378.12	451.6
VLASelect	46.06	35.41	39.46	64.71
	44.24	25.63	25.57	45.18

Overall, VLASelect *improves accuracy by an average of 40.12% under the new tasks, environment changes, and available resources fluctuations.*

5.3 Comparison of Overheads in Online RL

In this evaluation, we compare the overheads of VLASelect and baseline techniques under the same accuracy. For each technique, its overheads are breakdown into sampling and training stages of online RL.

5.3.1 Overheads Under The Same Accuracy.

Settings. We measure the overheads of VLASelect and baselines using two steps. First, we measure VLASelect’s overheads when it reaches its highest accuracy for each new task or environment changes in a workload. Second, we train each baseline method until it reaches the same accuracy as VLASelect. The evaluation consider three *overhead metrics*: training time (hour), memory footprint (GB) and energy consumption (kJ). We measure the memory footprint using the *jtop* command and the energy consumption using the REMD-10AL monitor.

Results. We report three types of results for each workload: (i) the average overhead across all tasks and environments (Table 2); (ii) the memory footprint curves in one new task (Figure 9); and (iii) the average energy consumption in one new task or one environment change (Table 3). We can see VLASelect consistently maintains the lowest overhead across four workloads for two reasons. First, it accelerates the training convergence by swapping in the large model’s most accuracy-related neurons. Second, it reduces the memory footprint by updating the large model through the neuron index without directly retraining. In contrast, some baseline methods exhibit particularly large overhead. For example, VLA-RFT and World-Env take 44.4% more memory footprint than other baselines, because they introduce a large world model and perform frequent forward operations on it.

Overall, VLASelect *reduces time cost, memory footprint, and energy consumption by an average of 11.49x, 1.34x (25.6%), and 11.55x respectively under the same accuracy.*

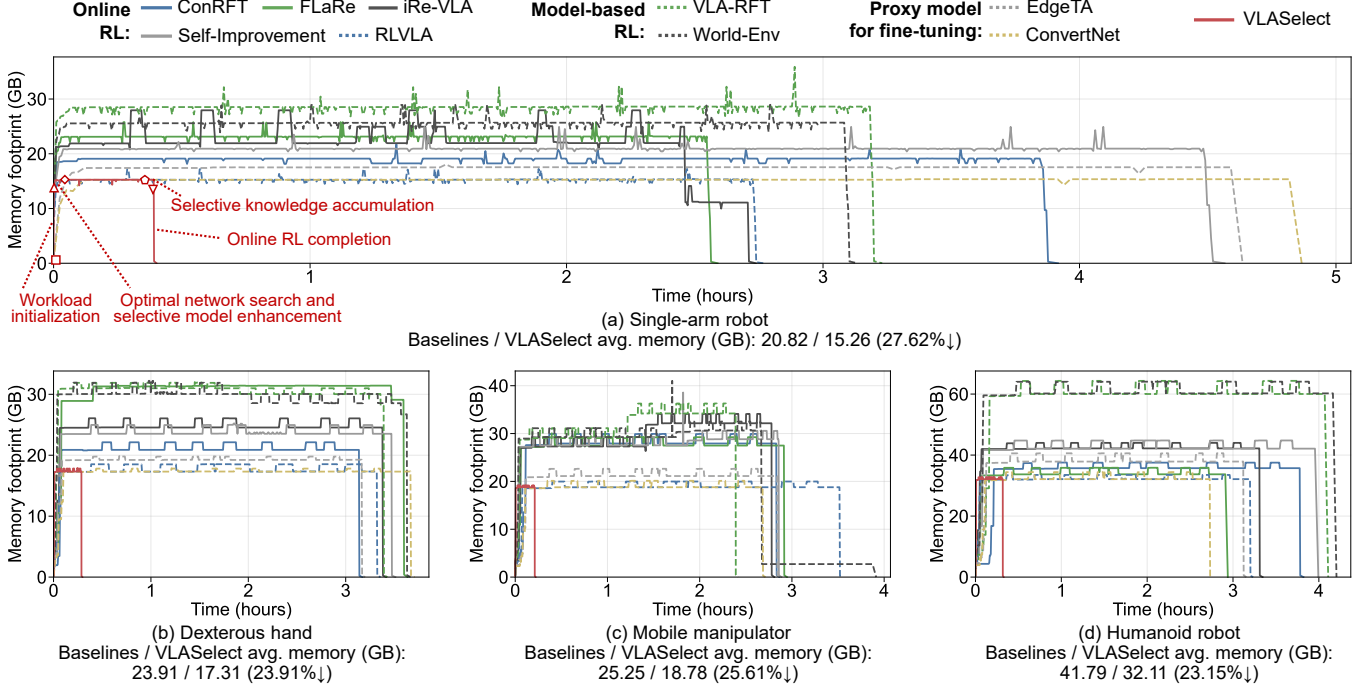


Figure 9. Memory footprint comparison in one new task.

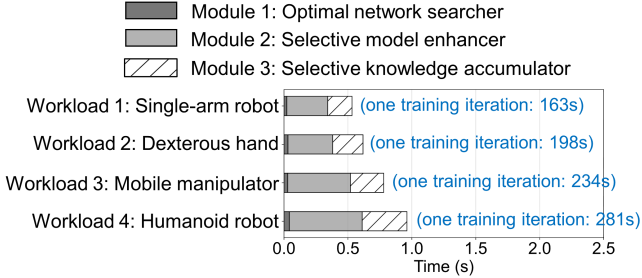


Figure 10. Time breakdown of VLASelect's modules.

5.3.2 Overhead Breakdown.

Settings. As listed in Table 2, We break down the training time for baseline and VLASelect. First, we measure the time of sampling and training for all techniques. We employ CUDA timers `torch.cuda.Event()` before and after each sampling and training, and measure the GPU wall-clock time by `event.elapsed_time()`. Second, we use the same approach to measure the execution time of VLASelect's three modules.

Results. Figure 11 shows the training time breakdown in each workload. We can see for all techniques, they spend a large proportion of time in sampling and baseline techniques have significantly longer sampling time. This is because existing techniques employ fixed models in exploration. The insufficient learning capacity of these models results in lower sample efficiency and longer sampling time (that is, more samples to reach the same accuracy as VLASelect).

Figure 10 illustrates the wall-clock time of VLASelect's three modules in each workload. We can see that for all workloads, these modules complete within 1s (less than $< 1\%$ of the total training time). In particular, the optimal network searcher is triggered at each training iteration and this module takes the least time (less than 0.05s). This guarantees that VLASelect can perform the network search and neuron swapping with small overheads, and thus respond promptly to environment changes.

5.4 Design Choice Validation by Ablation

Following the setup in Section 5.2.2, this evaluation tests the single-arm robot workload, and reports the learning accuracy under different design choices.

Scaling law function. Figure 12(a) shows that the proposed scaling law function for online RL brings 18.73% higher learning accuracy than the traditional scaling law for supervised pre-training. This is because this function has a higher estimation precision, thus guiding the network searcher towards a better network architecture.

Neuron-grained scaling. Figure 12(b) shows that scaling up with the most accuracy-related neurons from the large model achieves 25.81% higher accuracy than scaling up with randomly selected neurons or the most accuracy-unrelated neurons. This indicates that the most accuracy-related neurons are indeed critical to effective small-model scaling.

Moreover, Figure 12(c) shows that scaling down by freezing neurons improves accuracy by 47.01% compared with

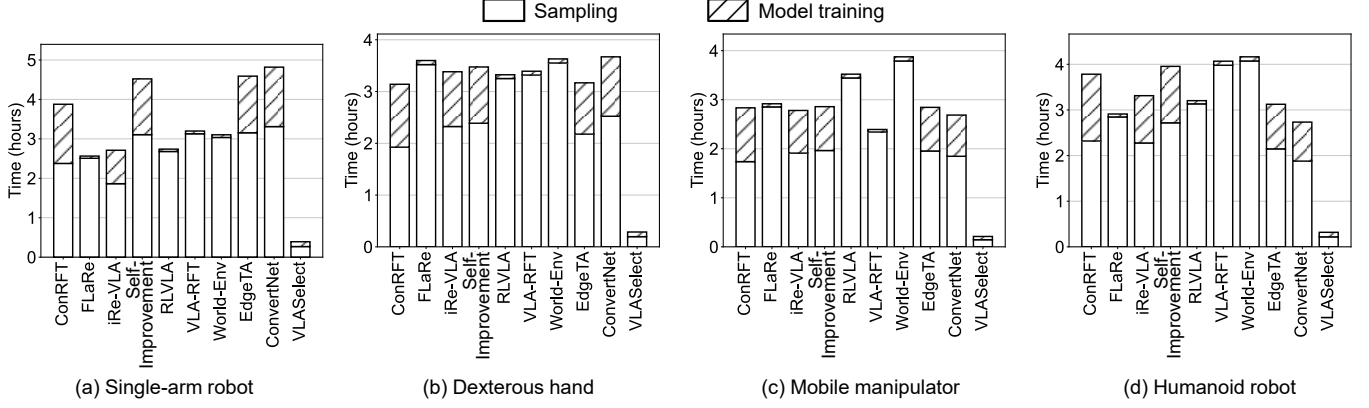


Figure 11. Time breakdown of sampling and training for all methods.

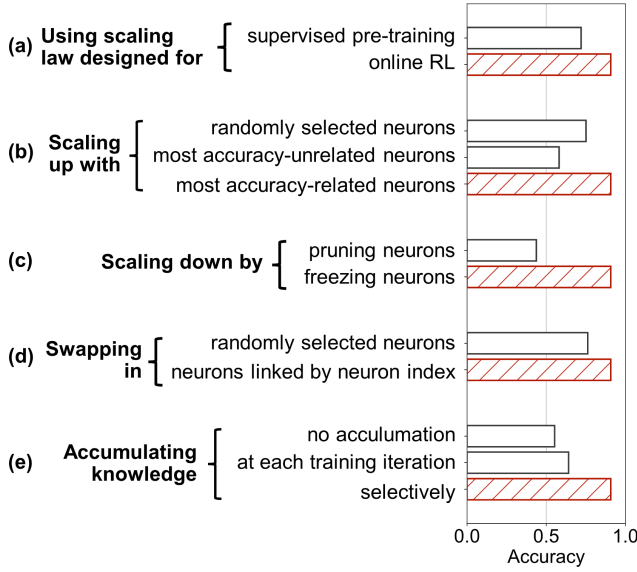


Figure 12. Accuracies of VLASelect under different design choices.

pruning neurons. This is because pruning causes immediate accuracy loss and requires extra retraining to recover [24].

Neuron swapping. Figure 12(d) shows that neuron swapping through neuron indexes achieves 14.55% higher accuracy than swapping in randomly selected neurons. This indicates that our neuron swapping approach can identify the large model’s neurons that best match the small model’s neuron, thus effectively improving the small model’s learning ability.

Knowledge accumulation. Figure 12(e) shows that the selective knowledge accumulation achieves a 31.83% higher accuracy than no accumulation or accumulation at each training iteration. This result highlights the effectiveness of our selective and positive knowledge transfer mechanism.

5.5 Discussions and Limitations

Sim-to-real transfer. We select robotic arm and dexterous hand to evaluate the simulation to reality transfer, as shown in Figure 13. For each robot, we construct 10 different task/environment settings, including picking up or grasping different objects under different illumination conditions. The results in Table 4 show that VLASelect has small accuracy decreases (5.70% on average) sim-to-real transfer. This is because its selective neuron-exchange mechanism also helps transfer the simulated model’s knowledge into the model in real world.

ICL. We take the single-arm robot workload as an example and compare the accuracy of VLASelect with a typical ICL method, RICL [54]. The results show that RICL achieves 38.13% lower accuracy on the first new task and 49.74% lower accuracy on the fifth new task. The result indicates that ICL indeed struggles for VLA models, which need to respond to dynamic environment with a sequence of robotic actions.

Applicability to VLA models. The current implementation of VLASelect is built upon online RL. Hence VLASelect is applicable to a majority of VLA models such as autoregressive VLA models [7, 60] (e.g. OpenVLA [27] and VLA-Adapter [59]), whose training use online RL. For flow-based VLA models (e.g. π_0 [6]), VLASelect need modifications of their training algorithms to fit the online RL framework [8].

Maximum supported model size. Using VLA-Adapter as an example, our evaluation result shows that VLASelect can support model size of up to 11.3GB, 11.3GB, and 24.0GB on NVIDIA AGX Xavier, NVIDIA AGX Orin (32GB), and NVIDIA AGX Orin (64GB), respectively. This is because VLASelect updates the large model by neuron swapping rather than resource expensive backward operations. Therefore, its maximum supported model size depends on the devices’ memory and computational capacities as well as the large model’s compression ratio (e.g. 2.12% for VLA-Adapter).

Applicability to multi-agent scenarios. VLASelect can be extended to multi-agent collaboration scenarios where

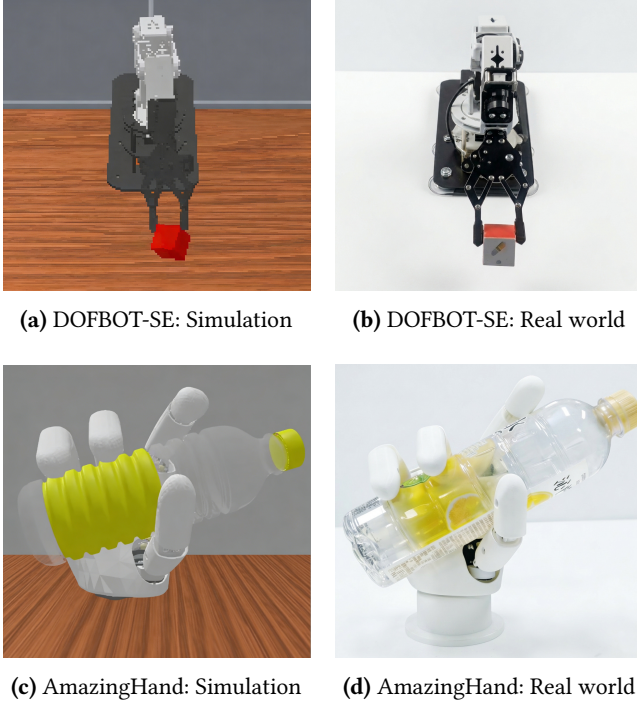


Figure 13. Robotic arm and dexterous hand used in sim-to-real evaluation.

each agent learns a sub-task of a complex workflow [42]. The basic idea is to extend our neuron swapping mechanism to multiple agents’ knowledge transfer. Such fine-grained knowledge differentiation allows VLASelect to extract specific neurons for each agent’s actions (i.e. output of its VLA model) and enable positive knowledge transfer among multiple agents’ similar actions. To validate this idea, we select a dual-arm robot, in which the two arms collaborate to pick up an object, and the initial result shows that VLASelect improves accuracy by 17.3%.

6 Related Work

There has been extensive works seeking to improve the performance of self-evolving agents by adjusting their non-parametric components, including ICL [12], prompt optimization [43, 48, 57, 61], tools [21, 45, 46], architectures [65, 70], and model inference strategy [52, 64]. They are mainly designed for LLMs and lack support for VLA models, which map visual and language inputs to robotic actions [7, 27, 59, 60]. Online and model-based RL techniques developed for VLA agents are mainly proposed in recent two years.

Online RL autonomously explores the environment to collect training samples and update model parameters. iRe-VLA [19] and SimpleVLA-RL [30] address the instability of training VLA models with online RL. RL-VLA [35], VLA-RL [36] and RLinf-VLA [68] develop benchmarks and open-source frameworks to improve model generalization.

Table 4. Accuracies of two robots in Sim-to-real transfer.

Task/ Environment	DOFBOT-SE		AmazingHand	
	Simulation	Real world	Simulation	Real world
1	83.00%	15/20	83.00%	15/20
2	78.00%	14/20	93.00%	17/20
3	86.00%	17/20	78.00%	15/20
4	85.00%	16/20	92.00%	18/20
5	83.00%	15/20	69.00%	13/20
6	72.00%	13/20	89.00%	16/20
7	68.00%	12/20	77.00%	14/20
8	85.00%	16/20	70.00%	13/20
9	78.00%	15/20	95.00%	17/20
10	88.00%	16/20	72.00%	14/20
(Average)	80.60%	149/200	81.80%	152/200

Model-Based RL constructs world models [33] and uses them to replace real interactive environment in RL [29]. For example, World-Env [62] generates a vision-based world model to train robot arms. However, existing RL techniques are mainly developed for fixed network architectures.

Proxy model for fine tuning [71–73] extracts the most accuracy-related parts of an original model, and transforms them into a compressed proxy model. Although training the proxy model reduces computational costs, its network architecture is determined by a given input dataset and has no adaptation to the dynamic environment.

Relationship to continuous learning in embodied AI. Some recent techniques study curriculum learning techniques [31, 44] and lifelong/continual/incremental learning techniques [23, 34, 69] for robotic tasks. They passively acquire knowledge from given task sequences, whereas VLASelect actively explores the environment to guide the agent’s learning trajectory.

7 Conclusion

This paper presents the design, implementation and evaluation of VLASelect, a system that enables large-small model collaborative learning for a self-evolving agent. Our approach can quickly explore the environment using a small model and selectively transfer the learned knowledge to the agent’s large model, while compensating the small model’s learning ability by swapping the large model’s most accuracy-related neurons. Extensive evaluation on realistic scenarios against online and model-based RL techniques prove the efficacy and practicality of VLASelect. Our future work will explore the collaborative learning of multiple self-evolving agents in the large-small model framework. Such co-learning allows an agent to learn from multiple agents’ local VLA models to increase its accuracy and generalization.

References

- [1] 2025. DOFBOT-SE single-arm robot. https://www.yahboom.net/study/DOFBOT_SE.
- [2] 2026. AmazingHand. <https://github.com/pollen-robotics/AmazingHand>.
- [3] 2026. Fetch mobile manipulator. <https://www.wevolver.com/specs/fetch.mobile.manipulator>.
- [4] 2026. Unitree Go1. <https://www.unitree.com/go1>.
- [5] Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. 2010. A theory of learning from different domains. *Machine learning* 79, 1 (2010), 151–175.
- [6] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y Galliker, et al. 2025. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. In *CoRL*’25.
- [7] Pawel Budzianowski, Wesley Maa, Matthew Freed, Jingxiang Mo, Winston Hsiao, Aaron Xie, Tomasz Młoduchowski, Viraj Tipnis, and Benjamin Bolte. 2025. Edgevla: Efficient vision-language-action models. *arXiv preprint arXiv:2507.14049* (2025).
- [8] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Quanlu Zhang, Zhaoqi Yu, Guoliang Fan, et al. 2025. π RL: Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889* (2025).
- [9] Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. 2025. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450* (2025).
- [10] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. 2022. Knowledge neurons in pretrained transformers. In *ACL*’22. 8493–8502.
- [11] Shibhansh Dohare, J Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A Rupam Mahmood, and Richard S Sutton. 2024. Loss of plasticity in deep continual learning. *Nature* 632, 8026 (2024), 768–774.
- [12] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, et al. 2024. A survey on in-context learning. In *EMNLP*’24. 1107–1128.
- [13] Gabriel Dulac-Arnold, Nir Levine, Daniel J. Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. 2021. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning* 110, 9 (2021), 2419–2468.
- [14] Mohamed Elsayed and A Rupam Mahmood. 2024. Addressing Loss of Plasticity and Catastrophic Forgetting in Continual Learning. In *ICLR*’24.
- [15] Junming Fan, Yue Yin, Tian Wang, Wenhong Dong, Pai Zheng, and Lihui Wang. 2025. Vision-language model-based human-robot collaboration for smart manufacturing: A state-of-the-art survey. *Frontiers of Engineering Management* 12, 1 (2025), 177–200.
- [16] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, et al. 2025. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407* (2025).
- [17] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. 2025. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. *arXiv preprint arXiv:2507.21046* (2025).
- [18] Xitong Gao, Yiren Zhao, Łukasz Dudziak, Robert Mullins, and Chengzhong Xu. 2019. Dynamic Channel Pruning: Feature Boosting and Suppression. In *ICLR*’19.
- [19] Yanjiang Guo, Jianke Zhang, Xiaoyu Chen, Xiang Ji, Yen-Jen Wang, Yucheng Hu, and Jianyu Chen. 2025. Improving vision-language-action model with online reinforcement learning. In *ICRA*’25. 15665–15672.
- [20] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149* (2015).
- [21] Mohd Ariful Haque, Justin Williams, Sunzida Siddique, Md Hujaifa Islam, Hasmat Ali, Kishor Datta Gupta, and Roy George. 2025. Advanced tool learning and selection system (atlass): A closed-loop framework using llm. In *SOSE*’25. 64–73.
- [22] Jiaheng Hu, Rose Hendrix, Ali Farhadi, Aniruddha Kembhavi, Roberto Martin-Martin, Peter Stone, Kuo-Hao Zeng, and Kiana Ehsani. 2025. Flare: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning. In *ICRA*’25. 3617–3624.
- [23] Jiaheng Hu, Jay Shim, Chen Tang, Yoonchang Sung, Bo Liu, Peter Stone, and Roberto Martin-Martin. 2026. Simple Recipe Works: Vision-Language-Action Models are Natural Continual Learners with Reinforcement Learning. *arXiv preprint arXiv:2603.11653* (2026).
- [24] Kai Huang, Boyuan Yang, and Wei Gao. 2023. Elastictrainer: Speeding up on-device training with runtime elastic tensor selection. In *MobiSys*’23. 56–69.
- [25] Nick Jakobi, Phil Husbands, and Inman Harvey. 1995. Noise and the Reality Gap: The Use of Simulation in Evolutionary Robotics. In *ECAL*’95. 704–720.
- [26] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [27] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, et al. 2025. OpenVLA: An Open-Source Vision-Language-Action Model. In *CoRL*’25. 2679–2713.
- [28] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. 2021. RMA: Rapid Motor Adaptation for Legged Robots. In *RSS*’21.
- [29] Hengtao Li, Pengxiang Ding, Runze Suo, Yihao Wang, Zirui Ge, Dongyuan Zang, Kexian Yu, Mingyang Sun, Hongyin Zhang, Donglin Wang, et al. 2025. Vla-rft: Vision-language-action reinforcement fine-tuning with verified rewards in world simulators. *arXiv preprint arXiv:2510.00406* (2025).
- [30] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. 2025. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674* (2025).
- [31] Pengna Li, Kangyi Wu, Shaoqing Xu, Fang Li, Hanbing Li, Lin Zhao, Kailin Lyu, Long Chen, Zhi-Xin Yang, and Nanning Zheng. 2026. SpaAct: Spatially-Activated Transition Learning with Curriculum Adaptation for Vision-Language Navigation. *arXiv preprint arXiv:2604.27620* (2026).
- [32] Shuai Li, Yizhe Chen, Dong Li, Sichao Liu, Dapeng Lan, Yu Liu, and Zhibo Pang. 2025. Transferring Vision-Language-Action Models to Industry Applications: Architectures, Performance, and Challenges. In *IAI*’25. 1–6.
- [33] Xinqing Li, Xin He, Le Zhang, Min Wu, Xiaoli Li, and Yun Liu. 2025. A comprehensive survey on world models for embodied ai. *arXiv preprint arXiv:2510.16732* (2025).
- [34] Huihan Liu, Changyeon Kim, Bo Liu, Minghuan Liu, and Yuke Zhu. 2026. Pretrained Vision-Language-Action Models are Surprisingly Resistant to Forgetting in Continual Learning. *arXiv preprint arXiv:2603.03818* (2026).
- [35] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. 2026. What can rl bring to vla generalization? an empirical study. *Advances in Neural Information Processing Systems* 38 (2026), 97121–97151.
- [36] Guanxing Lu, Wenkai Guo, Chubin Zhang, Yuheng Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. 2025. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719* (2025).

- [37] Ekdeep Singh Lubana and Robert P Dick. 2021. A Gradient Flow Framework For Analyzing Network Pruning. In *ICLR'21*.
- [38] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems* 36 (2023), 21702–21720.
- [39] Yuen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. 2026. A Survey on Vision–Language–Action Models for Embodied AI. *IEEE Transactions on Neural Networks and Learning Systems* (2026).
- [40] Oier Mees, Dibya Ghosh, Karl Pertsch, Kevin Black, Homer Rich Walke, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, et al. 2024. Octo: An open-source generalist robot policy. In *VLMNM Workshop @ ICRA'24*.
- [41] Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. 2017. Pruning Convolutional Neural Networks for Resource Efficient Inference. In *ICLR'17*.
- [42] James Orr and Ayan Dutta. 2023. Multi-agent deep reinforcement learning for multi-robot applications: A survey. *Sensors* 23, 7 (2023), 3625.
- [43] Dengyun Peng, Yuhang Zhou, Qiguang Chen, Jinhao Liu, Jingjing Chen, Libo Qin, and Wanxiang Che. 2025. Dlpo: Towards a robust, efficient, and generalizable prompt optimization framework from a deep-learning perspective. *arXiv preprint arXiv:2503.13413* (2025).
- [44] Bangyu Qin, Yue Gao, and Yi Bai. 2019. Sim-to-real: Six-legged robot control with deep reinforcement learning and curriculum learning. In *ICRAE'19*, 1–5.
- [45] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, et al. 2025. Alita: Generalist agent enabling scalable agentic reasoning with minimal pre-definition and maximal self-evolution. *arXiv preprint arXiv:2505.20286* (2025).
- [46] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2025. From exploration to mastery: Enabling llms to master tools via self-driven interactions. In *ICLR'25*.
- [47] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. 2018. Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations. In *RSS'18*.
- [48] Kiran Ramnath, Kang Zhou, Sheng Guan, Soumya Smruti Mishra, Xuan Qi, Zhengyuan Shen, Shuai Wang, Sangmin Woo, Sullam Jeoung, Yawei Wang, et al. 2025. A systematic survey of automatic prompt optimization techniques. In *EMNLP'25*, 33066–33098.
- [49] Ranjan Sapkota, Yang Cao, Konstantinos I Roumeliotis, and Manoj Karkee. 2025. Vision-language-action models: Concepts, progress, applications and challenges. *arXiv preprint arXiv:2505.04769* (2025).
- [50] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [51] Seyed Kamyar Seyed Ghasemipour, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, and Igor Mordatch. 2026. Self-Improving Embodied Foundation Models. *Advances in Neural Information Processing Systems* 38 (2026), 112651–112686.
- [52] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).
- [53] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *SIGOPS'24*, 590–606.
- [54] Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. 2025. Ricl: Adding in-context adaptability to pre-trained vision-language-action models. *arXiv preprint arXiv:2508.02062* (2025).
- [55] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, et al. 2024. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425* (2024).
- [56] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. 2017. Domain randomization for transferring deep neural networks from simulation to the real world. In *IROS'17*, 23–30.
- [57] Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Haotian Luo, Jiayou Zhang, Nebojsa Jojic, Eric Xing, and Zhiting Hu. 2024. Promptagent: Strategic planning with language models enables expert-level prompt optimization. In *ICLR'24*.
- [58] Xiaozhi Wang, Kaiyue Wen, Zhengyan Zhang, Lei Hou, Zhiyuan Liu, and Juanzi Li. 2022. Finding skill neurons in pre-trained transformer-based language models. In *EMNLP'22*, 11132–11152.
- [59] Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, et al. 2026. Vla-adapter: An effective paradigm for tiny-scale vision-language-action model. In *AAAI'26*, Vol. 40, 18638–18646.
- [60] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. 2025. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters* (2025).
- [61] Jinyu Xiang, Jiayi Zhang, Zhaoyang Yu, Xinbing Liang, Fengwei Teng, Jinhao Tu, Fashen Ren, Xiangru Tang, Sirui Hong, Chenglin Wu, et al. 2025. Self-Supervised Prompt Optimization. *arXiv preprint arXiv:2502.06855* (2025).
- [62] Junjin Xiao, Yandan Yang, Xinyuan Chang, Ronghan Chen, Feng Xiong, Mu Xu, Wei-Shi Zheng, and Qing Zhang. 2025. World-env: Leveraging world model as a virtual environment for vla post-training. *arXiv preprint arXiv:2509.24948* (2025).
- [63] Xinyu Xu, Yizheng Zhang, Yong-Lu Li, Lei Han, and Cewu Lu. 2024. Humanvla: Towards vision-language directed object rearrangement by physical humanoid. *Advances in Neural Information Processing Systems* 37 (2024), 18633–18659.
- [64] Wenkai Yang, Shuming Ma, Yankai Lin, and Furu Wei. 2026. Towards thinking-optimal scaling of test-time compute for llm reasoning. *Advances in Neural Information Processing Systems* 38 (2026), 43605–43631.
- [65] Rui Ye, Xiangrui Liu, Qimin Wu, Xianghe Pang, Zhenfei Yin, Lei Bai, and Siheng Chen. 2025. X-mas: Towards building multi-agent systems with heterogeneous llms. *arXiv preprint arXiv:2505.16997* (2025).
- [66] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems* 7 (2025).
- [67] Shixing Yu, Zhewei Yao, Amir Gholami, Zhen Dong, Sehoon Kim, Michael W Mahoney, and Kurt Keutzer. 2022. Hessian-aware pruning and optimal neural implant. In *WACV'22*, 3880–3891.
- [68] Hongzhi Zang, Mingjie Wei, Si Xu, Yongji Wu, Zhen Guo, Yuanqing Wang, Hao Lin, Liangzhi Shi, Yuqing Xie, Zhexuan Xu, et al. 2025. Rlinf-vla: A unified and efficient framework for vla+ rl training. *arXiv preprint arXiv:2510.06710* (2025).
- [69] Qixin Zeng, Shuo Zhang, Hongyin Zhang, Renjie Wang, Han Zhao, Libang Zhao, Runze Li, Donglin Wang, and Chao Huang. 2026. CRL-VLA: Continual Vision-Language-Action Learning. *arXiv preprint arXiv:2602.03445* (2026).
- [70] Guibin Zhang, Kaijie Chen, Guancheng Wan, Heng Chang, Hong Cheng, Kun Wang, Shuyue Hu, and Lei Bai. 2025. Evoflow: Evolving diverse agentic workflows on the fly. *arXiv preprint arXiv:2502.07373* (2025).
- [71] Qinglong Zhang, Rui Han, Chi Harold Liu, Guoren Wang, and Lydia Y Chen. 2024. ElasticDNN: On-device neural network remodeling for adapting evolving vision domains at edge. *IEEE Trans. Comput.* 73, 6 (2024), 1616–1630.
- [72] Qinglong Zhang, Rui Han, Chi Harold Liu, Guoren Wang, and Lydia Y Chen. 2026. ConvertNet: Training-time Model Scaling for In-vehicle

- Continuous Learning. *IEEE Transactions on Mobile Computing* (2026).
- [73] Qinglong Zhang, Rui Han, Chi Harold Liu, Guoren Wang, Song Guo, and Lydia Y Chen. 2024. Edgeta: Neuron-grained scaling of foundation models in edge-side retraining. *IEEE Transactions on Mobile Computing* 24, 4 (2024), 2690–2707.